

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 1, course “Discrete Analysis”

Student: M. A. Khorokhorin
Teacher: A. A. Kukhtichev
Group: M8O-206BV-24
Date: 05.03.2026
Grade:
Signature:

Moscow, 2026

Laboratory work No. 1

J. 4-1	
Time limit	3 seconds
Memory limit	300 MB
Input	standard input or <code>input.txt</code>
Output	standard output or <code>output.txt</code>

Task: Implement a program that reads key-value pairs, sorts them by key in ascending order using a linear-time sorting algorithm, and outputs the sorted sequence. The variant is defined by the key type (and the corresponding sorting method) and the value type.

Sorting method: radix sort.

Key type: dates in the format DD.MM.YYYY, e.g. 1.1.1, 1.9.2009, 01.09.2009, 31.12.2009.

Value type: fixed-length strings of 64 characters. Input strings may be shorter; in that case they are padded with null characters up to 64, which are not printed.

Input format

Each non-empty line of the input contains a key-value pair: the key as specified above, followed by a tab character, followed by the corresponding value.

Output format

The output contains the same lines as the input (excluding empty ones), in sorted order.

Example

Input:

```
1.1.1      n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naatt
01.02.2008 n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naat
1.1.1      n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naa
01.02.2008 n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3na
```

Output:

```
1.1.1      n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naatt
1.1.1      n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naa
01.02.2008 n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3naat
01.02.2008 n399tann9nnt3ttnaaan9nann93na9t3a3t9999na3aan9antt3tn93aat3na
```

1 Description

The task is to implement the radix sort algorithm for “date–value” pairs.

The main idea of radix sort is to split every input element into digits and to apply a stable counting sort to each digit, from the least significant digit to the most significant one. After all digits have been processed, the array is fully sorted. The complexity of the algorithm is $O(n \cdot k)$, where k is the number of digits and n is the number of elements.

2 Source code

1 Auxiliary structures

The key is a date in the format `DD.MM.YYYY`, which can be uniquely represented as an integer by the formula $year \times 10000 + month \times 100 + day$. This encoding preserves the order of dates under numeric comparison.

Since the number of input lines is not known in advance, a dynamic array `Vector<T>` that doubles its capacity on overflow is implemented. It follows the rule of five: destructor, copy constructor, copy assignment operator, move constructor and move assignment operator. The structure `Pair` is a plain aggregate of a `std::string` and a `std::uint32_t` and follows the rule of zero: all special member functions are generated by the compiler.

Input is read line by line. Blank lines and lines consisting only of whitespace are skipped, and a trailing carriage return is stripped. A line without a tab separator, a value longer than 64 characters or an invalid date is rejected: the program prints a message starting with `ERROR:` to the standard error stream and exits with code 1.

include/vector.hpp	
<code>template <typename T> class Vector</code>	Dynamic array with the rule of five; methods <code>pushBack</code> , <code>size</code> , <code>capacity</code> , <code>empty</code> , <code>data</code> , <code>begin</code> , <code>end</code> , <code>swap</code> , <code>operator[]</code>
include/pair.hpp, src/pair.cpp	
<code>struct Pair</code>	Key-value pair: stores the original line <code>raw</code> and the numeric key <code>key</code> of type <code>std::uint32_t</code>
<code>bool isBlank(std::string_view)</code>	Checks whether a line consists only of whitespace characters
<code>std::uint32_t parseDateKey(std::string_view)</code>	Parses a date <code>DD.MM.YYYY</code> and encodes it as an integer key
<code>Pair parsePair(std::string)</code>	Parses one input line into a pair
<code>Vector<Pair> readPairs(std::istream&)</code>	Reads all non-blank lines of the input
<code>void writePairs(std::ostream&, const Vector<Pair>&)</code>	Prints the pairs in the original form
include/radix_sort.hpp, src/radix_sort.cpp	
<code>void countingPass(Vector<Pair>&, Vector<Pair>&, int)</code>	One counting sort pass over the given byte of the key
<code>void radixSort(Vector<Pair>&)</code>	Radix sort: four passes over the bytes of <code>std::uint32_t</code>

Listing 1: include/vector.hpp

```

1  #pragma once
2
3  #include <cstddef>
4  #include <utility>
5
6  template <typename T>
7  class Vector {
8  public:
9      Vector() = default;
10
11     explicit Vector(std::size_t count)
12         : data_(count > 0 ? new T[count] : nullptr), size_(count), capacity_(count) {}
13
14     Vector(const Vector& other)
15         : data_(other.capacity_ > 0 ? new T[other.capacity_] : nullptr),
16           size_(other.size_),
17           capacity_(other.capacity_) {
18         for (std::size_t i = 0; i < size_; ++i) {
19             data_[i] = other.data_[i];
20         }
21     }
22
23     Vector& operator=(const Vector& other) {
24         if (this != &other) {
25             Vector copy(other);
26             swap(copy);
27         }
28         return *this;
29     }
30
31     Vector(Vector&& other) noexcept
32         : data_(std::exchange(other.data_, nullptr)),
33           size_(std::exchange(other.size_, 0)),
34           capacity_(std::exchange(other.capacity_, 0)) {}
35
36     Vector& operator=(Vector&& other) noexcept {
37         if (this != &other) {
38             delete[] data_;
39             data_ = std::exchange(other.data_, nullptr);
40             size_ = std::exchange(other.size_, 0);
41             capacity_ = std::exchange(other.capacity_, 0);
42         }
43         return *this;
44     }
45
46     ~Vector() { delete[] data_; }
47
48     void swap(Vector& other) noexcept {
49         std::swap(data_, other.data_);
50         std::swap(size_, other.size_);
51         std::swap(capacity_, other.capacity_);
52     }
53
54     [[nodiscard]] std::size_t size() const { return size_; }
55     [[nodiscard]] std::size_t capacity() const { return capacity_; }
56     [[nodiscard]] bool empty() const { return size_ == 0; }
57
58     T* data() { return data_; }
59     const T* data() const { return data_; }

```

```

60
61     T* begin() { return data_; }
62     T* end() { return data_ + size_; }
63     const T* begin() const { return data_; }
64     const T* end() const { return data_ + size_; }
65
66     void pushBack(T value) {
67         if (size_ == capacity_) {
68             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
69         }
70         data_[size_++] = std::move(value);
71     }
72
73     T& operator[](std::size_t index) { return data_[index]; }
74     const T& operator[](std::size_t index) const { return data_[index]; }
75
76 private:
77     void reallocate(std::size_t newCapacity) {
78         T* newData = new T[newCapacity];
79         for (std::size_t i = 0; i < size_; ++i) {
80             newData[i] = std::move(data_[i]);
81         }
82         delete[] data_;
83         data_ = newData;
84         capacity_ = newCapacity;
85     }
86
87     T* data_ = nullptr;
88     std::size_t size_ = 0;
89     std::size_t capacity_ = 0;
90 };

```

Listing 2: include/pair.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <iosfwd>
5  #include <string>
6  #include <string_view>
7
8  #include "vector.hpp"
9
10 inline constexpr std::size_t VALUE_LENGTH = 64;
11
12 struct Pair {
13     std::string raw;
14     std::uint32_t key = 0;
15 };
16
17 bool isBlank(std::string_view line);
18 std::uint32_t parseDateKey(std::string_view date);
19 Pair parsePair(std::string line);
20 Vector<Pair> readPairs(std::istream& input);
21 void writePairs(std::ostream& output, const Vector<Pair>& pairs);

```

Listing 3: src/pair.cpp

```

1  #include "pair.hpp"

```

```

2
3 #include <algorithm>
4 #include <charconv>
5 #include <cstdint>
6 #include <cstdint>
7 #include <istream>
8 #include <ostream>
9 #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <system_error>
13 #include <utility>
14
15 namespace {
16
17 constexpr std::uint32_t MAX_DAY = 31;
18 constexpr std::uint32_t MAX_MONTH = 12;
19 constexpr std::uint32_t MAX_YEAR = 9999;
20 constexpr std::uint32_t YEAR_WEIGHT = 10000;
21 constexpr std::uint32_t MONTH_WEIGHT = 100;
22
23 std::uint32_t parseNumber(std::string_view text) {
24     const bool allDigits =
25         std::ranges::all_of(text, [](char symbol) { return symbol >= '0' && symbol <= '9'; });
26     if (text.empty() || !allDigits) {
27         throw std::invalid_argument("invalid date");
28     }
29     std::uint32_t value = 0;
30     const auto [end, error] = std::from_chars(text.data(), text.data() + text.size(), value);
31     if (error != std::errc{} || end != text.data() + text.size()) {
32         throw std::invalid_argument("invalid date");
33     }
34     return value;
35 }
36
37 }
38
39 bool isBlank(std::string_view line) {
40     return std::ranges::all_of(line, [](char symbol) {
41         return symbol == ' ' || symbol == '\t' || symbol == '\r' || symbol == '\n' ||
42             symbol == '\v' || symbol == '\f';
43     });
44 }
45
46 std::uint32_t parseDateKey(std::string_view date) {
47     const std::size_t firstDot = date.find('.');
48     if (firstDot == std::string_view::npos) {
49         throw std::invalid_argument("invalid date");
50     }
51     const std::size_t secondDot = date.find('.', firstDot + 1);
52     if (secondDot == std::string_view::npos) {
53         throw std::invalid_argument("invalid date");
54     }
55     const std::uint32_t day = parseNumber(date.substr(0, firstDot));
56     const std::uint32_t month = parseNumber(date.substr(firstDot + 1, secondDot - firstDot - 1));
57     const std::uint32_t year = parseNumber(date.substr(secondDot + 1));
58     if (day < 1 || day > MAX_DAY || month < 1 || month > MAX_MONTH || year > MAX_YEAR) {
59         throw std::invalid_argument("invalid date");
60     }
61     return year * YEAR_WEIGHT + month * MONTH_WEIGHT + day;
62 }
63

```

```

64 Pair parsePair(std::string line) {
65     if (!line.empty() && line.back() == '\r') {
66         line.pop_back();
67     }
68     const std::size_t tab = line.find('\t');
69     if (tab == std::string::npos) {
70         throw std::invalid_argument("missing tab separator");
71     }
72     if (line.size() - tab - 1 > VALUE_LENGTH) {
73         throw std::invalid_argument("value is longer than 64 characters");
74     }
75     const std::uint32_t key = parseDateKey(std::string_view(line).substr(0, tab));
76     return Pair{.raw = std::move(line), .key = key};
77 }
78
79 Vector<Pair> readPairs(std::istream& input) {
80     Vector<Pair> pairs;
81     std::string line;
82     std::size_t lineNumber = 0;
83     while (std::getline(input, line)) {
84         ++lineNumber;
85         if (isBlank(line)) {
86             continue;
87         }
88         try {
89             pairs.pushBack(parsePair(std::exchange(line, {})));
90         } catch (const std::invalid_argument& error) {
91             throw std::invalid_argument("line " + std::to_string(lineNumber) + ": " + error.what());
92         }
93     }
94     return pairs;
95 }
96
97 void writePairs(std::ostream& output, const Vector<Pair>& pairs) {
98     for (const Pair& pair : pairs) {
99         output << pair.raw << '\n';
100     }
101 }

```

2 Counting sort over one byte

The function `countingPass` performs one stable pass over the given byte of the key. First, an array of frequencies is built (256 elements, one for each possible byte value), then it is transformed into an array of prefix sums. The elements are placed into the output array in reverse order, which guarantees stability.

3 Radix sort

The function `radixSort` applies `countingPass` successively to bytes 0, 1, 2, 3 (from the least significant to the most significant). To avoid repeated allocations, a preallocated `buffer` is used: after each pass the input array and the buffer are exchanged with `swap`, so no elements are copied back.

Listing 4: include/radix_sort.hpp

```

1 | #pragma once
2 |
3 | #include "pair.hpp"
4 | #include "vector.hpp"
5 |
6 | void countingPass(Vector<Pair>& input, Vector<Pair>& output, int byteIndex);
7 | void radixSort(Vector<Pair>& data);

```

Listing 5: src/radix_sort.cpp

```

1 | #include "radix_sort.hpp"
2 |
3 | #include <array>
4 | #include <cstdint>
5 | #include <cstdint>
6 | #include <utility>
7 |
8 | namespace {
9 |
10 | constexpr std::size_t BYTE_BASE = 256;
11 | constexpr std::uint32_t BYTE_MASK = 0xFF;
12 | constexpr int KEY_BYTES = 4;
13 | constexpr int BITS_PER_BYTE = 8;
14 |
15 | std::size_t byteAt(std::uint32_t key, int byteIndex) {
16 |     return (key >> (byteIndex * BITS_PER_BYTE)) & BYTE_MASK;
17 | }
18 |
19 | }
20 |
21 | void countingPass(Vector<Pair>& input, Vector<Pair>& output, int byteIndex) {
22 |     std::array<std::size_t, BYTE_BASE> count{};
23 |     for (const Pair& pair : input) {
24 |         ++count[byteAt(pair.key, byteIndex)];
25 |     }
26 |     for (std::size_t i = 1; i < BYTE_BASE; ++i) {
27 |         count[i] += count[i - 1];
28 |     }
29 |     for (std::size_t i = input.size(); i > 0; --i) {
30 |         Pair& pair = input[i - 1];
31 |         const std::size_t position = --count[byteAt(pair.key, byteIndex)];
32 |         output[position] = std::move(pair);
33 |     }
34 | }
35 |
36 | void radixSort(Vector<Pair>& data) {
37 |     if (data.size() < 2) {
38 |         return;
39 |     }
40 |     Vector<Pair> buffer(data.size());
41 |     for (int byteIndex = 0; byteIndex < KEY_BYTES; ++byteIndex) {
42 |         countingPass(data, buffer, byteIndex);
43 |         data.swap(buffer);
44 |     }
45 | }

```

4 Entry point

The function `main` reads the pairs, sorts them with `radixSort` and prints the result. Any parsing error is reported to the standard error stream.

Listing 6: `app/main.cpp`

```
1  #include <exception>
2  #include <iostream>
3
4  #include "pair.hpp"
5  #include "radix_sort.hpp"
6  #include "vector.hpp"
7
8  int main() {
9      std::ios::sync_with_stdio(false);
10     std::cin.tie(nullptr);
11
12     try {
13         Vector<Pair> pairs = readPairs(std::cin);
14         radixSort(pairs);
15         writePairs(std::cout, pairs);
16     } catch (const std::exception& error) {
17         std::cerr << "ERROR: " << error.what() << '\n';
18         return 1;
19     }
20     return 0;
21 }
```

3 Console

```
# make build PRESET=release
# build/release/lab1/lab1_main
01.01.2000    Alice
15.06.1995    Bob
03.03.2010    Charlie
20.11.1988    Diana
^D
20.11.1988    Diana
15.06.1995    Bob
01.01.2000    Alice
03.03.2010    Charlie
```

4 Performance test

1 Method

For the comparison, two identical arrays of randomly generated dates are created (`lab1/benchmark/benchmark.cpp`). One of them is sorted by the implemented `radixSort`, the other by the standard function `std::stable_sort` from the STL. The time of each algorithm is measured separately with `std::chrono::steady_clock`. Data generation and input/output are not included in the measurement.

The test is run for five sizes from 100 to 10^6 elements.

2 Results

N	<code>radixSort</code> , μs	<code>std::stable_sort</code> , μs
100	8	7
1 000	66	74
10 000	831	1 133
100 000	14 291	19 226
1 000 000	140 767	191 842

Table 1: Sorting time in microseconds

```
# make build PRESET=release
# python3 lab1/benchmark/generator.py 100 && \
    build/release/lab1/lab1_benchmark < lab1/benchmark/tests.txt
Radix sort time: 8us
STL stable sort time: 7us
# python3 lab1/benchmark/generator.py 1000 && \
    build/release/lab1/lab1_benchmark < lab1/benchmark/tests.txt
Radix sort time: 66us
STL stable sort time: 74us
# python3 lab1/benchmark/generator.py 10000 && \
    build/release/lab1/lab1_benchmark < lab1/benchmark/tests.txt
Radix sort time: 831us
STL stable sort time: 1133us
# python3 lab1/benchmark/generator.py 100000 && \
    build/release/lab1/lab1_benchmark < lab1/benchmark/tests.txt
Radix sort time: 14291us
STL stable sort time: 19226us
```

```
# python3 lab1/benchmark/generator.py 1000000 && \
  build/release/lab1/lab1_benchmark < lab1/benchmark/tests.txt
Radix sort time: 140767us
STL stable sort time: 191842us
```

3 Analysis of the results

For small inputs ($N = 100$) both algorithms finish in a fraction of a millisecond, and the overhead is comparable to the useful work, so the difference is negligible.

Starting from $N = 1\,000$, radix sort is consistently faster than `std::stable_sort`. This agrees with the theoretical estimates: the complexity of `radixSort` is $O(n \cdot k)$, where $k = 4$ is the fixed number of bytes in `std::uint32_t`, that is, effectively $O(n)$. The complexity of `std::stable_sort` is $O(n \log n)$.

For $N = 10^6$ radix sort is **1.36 times** faster (140 767 μs versus 191 842 μs). The gap is smaller than one might expect from a linear algorithm, because `radixSort` makes 4 full passes over the array (4 passes over memory in total), while `std::stable_sort` uses cache-friendly block operations. Nevertheless, the linear asymptotics is confirmed: when N grows 10 times, the time of `radixSort` grows about 10 times, while `std::stable_sort` grows slightly faster.

5 Work diary

- 05.03.2026: studied the task statement, chose the way to encode a date as an integer key and defined the format of the “key–string” pair.
- 05.03.2026: implemented the template dynamic array, the key-value pair and the stable radix sort based on counting sort over bytes.
- 05.03.2026: prepared the test generator and the benchmark, compared the sort with `std::stable_sort` on inputs of different sizes.
- 05.03.2026: checked edge cases: empty input, equal keys, minimal and maximal date values; wrote the report.

6 Conclusions

While doing this laboratory work for the course “Discrete Analysis”, I learned to implement the radix sort and counting sort algorithms. I figured out how to sort data byte by byte using the type `std::uint32_t` as the key, and how to compactly encode a date as a numeric value for efficient comparison. I also recalled how to write a template dynamic array and apply the rule of five.

References

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms, 2nd edition*. — MIT Press, 2001. — 1180 p.
- [2] *Radix sort* — *Wikipedia*.
URL: https://en.wikipedia.org/wiki/Radix_sort (accessed: 05.03.2026).